

Student Performance Prediction, and prediction.

Three architecture diagrams before writing any code.

Student Performance Prediction System

Project Goal

Predict a student's **Final Exam Marks** based on academic performance using a **Linear Regression Model**.

Inputs

- Attendance
- Study Hours
- Previous Marks
- Assignment Score
- Internal Assessment

Output

- Final Marks
-

Diagram 1 – Complete Machine Learning Workflow

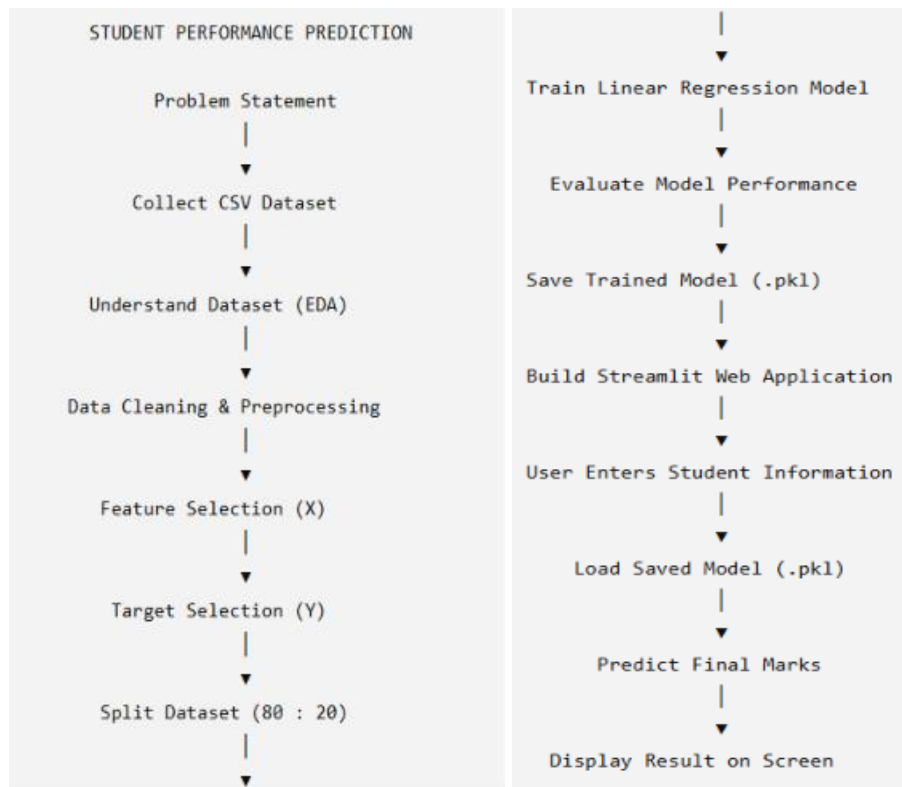
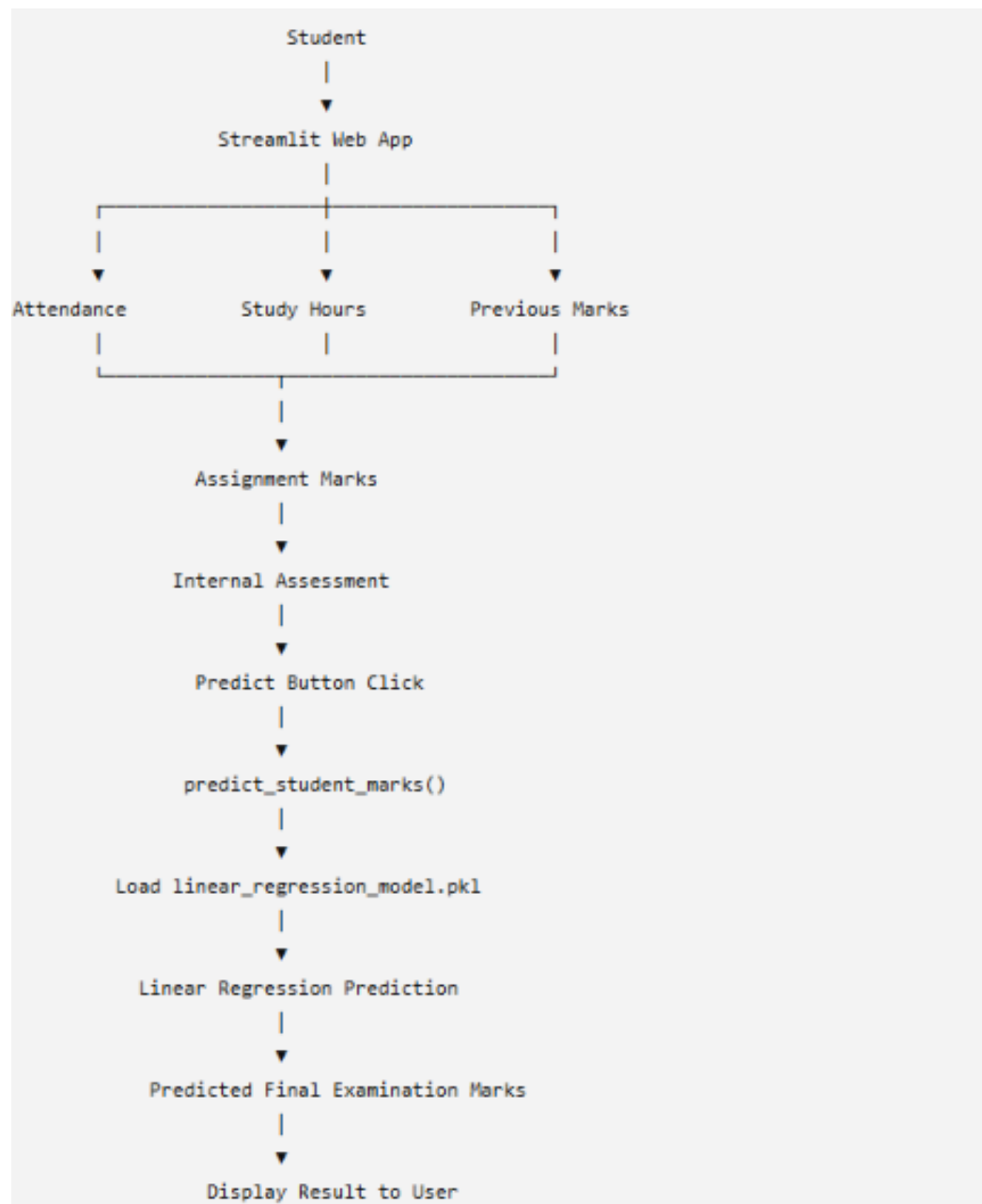


Diagram 2 – Project Folder Structure

```
StudentPerformancePrediction/
|
├─ app/
|   └─ app.py                # Streamlit User Interface
|
├─ data/
|   └─ student_performance.csv
|
├─ model/
|   └─ linear_regression_model.pkl
|
├─ src/
|   ├── __init__.py
|   ├── config.py
|   ├── preprocess.py
|   ├── train_model.py
|   ├── evaluate.py
|   └─ predict.py
|
├─ assets/
|
├─ requirements.txt
├─ README.md
└─ .gitignore
```

Diagram 3 – Project Architecture



Folder Responsibilities

Folder/File	Purpose
data/	Stores the CSV dataset
src/preprocess.py	Cleans and prepares the dataset
src/train_model.py	Trains the Linear Regression model
src/evaluate.py	Calculates MAE, MSE, RMSE, and R^2
model/	Stores the trained .pkl model
src/predict.py	Loads the model and predicts marks
app/app.py	Streamlit web application
config.py	Centralizes file paths and constants
requirements.txt	Lists required Python packages
README.md	Project documentation

Understanding Roadmap (Recommended Sequence)

Topic

- 1 Introduction to the Project & Problem Statement
- 2 Understanding the Dataset (EDA)
- 3 Data Preprocessing
- 4 Feature Selection & Target Variable
- 5 Train-Test Split
- 6 Building the Linear Regression Model
- 7 Model Evaluation
- 8 Saving the Model with Joblib

Topic

9 Making Predictions

10 Building the Streamlit Application

11 Testing the Application

12 Project Deployment

This gives you a complete view of the end-to-end workflow before they begin coding.

In the next step, we can design the **dataset architecture** itself—explaining every column, identifying the feature variables (X) and target variable (Y), and walking through the CSV so you understand exactly what the model learns before writing the first line of code.

Student Performance Prediction System

1. Introduction to the Project

Welcome to our first real Machine Learning project. Until now, we have learned Python programming, data manipulation using Pandas, visualization using Matplotlib and Seaborn, Machine Learning using Scikit-learn, and web application development using Streamlit. In this project, we will combine all these technologies to build a complete AI application.

Our project is called **Student Performance Prediction System**. The objective is to predict a student's final examination marks before the examination takes place. Instead of manually estimating student performance, we will train a Machine Learning model using historical student data and allow the computer to make predictions automatically.

This project follows the same development process used by software companies and AI engineers in industry.

2. Problem Statement

Every school or college wants to identify students who may score low marks so that teachers can provide additional guidance before the final examination.

Suppose a teacher has 500 students. Manually analyzing each student's attendance, study hours, assignment marks, and previous performance is difficult and time-consuming. Different teachers may also make different judgments.

Machine Learning solves this problem by learning patterns from previous student records and predicting the expected final marks of new students with consistency and speed.

Our goal is to develop a system that accepts student information and predicts the final examination marks automatically.

3. Project Workflow

Every Machine Learning project follows a standard lifecycle. Understanding this workflow is more important than writing code because every future AI project—whether it predicts house prices, medical insurance costs, or employee salaries—will follow the same process.

The workflow is:

Problem Statement → Dataset Collection → Data Analysis → Data Preprocessing → Feature Selection → Train-Test Split → Model Training → Model Evaluation → Save Model → Build Streamlit Application → Prediction → Deployment

This workflow is the backbone of our entire project.

4. Understanding the Dataset

Machine Learning cannot learn without data. The dataset is the foundation of our project.

Each row in the dataset represents one student.

Each column represents one characteristic of that student.

For example:

- Attendance
- Study Hours
- Previous Marks
- Assignment Marks
- Internal Assessment
- Final Marks

The first five columns are called **features** because they describe the student.

The last column, **Final Marks**, is called the **target variable** because this is what we want our Machine Learning model to predict.

Before training the model, we always explore the dataset using Pandas to understand its structure, number of rows, number of columns, missing values, and statistical information. This process is called **Exploratory Data Analysis (EDA)**.

5. Data Preprocessing

Real-world data is rarely perfect. Some records may contain missing values, incorrect data types, duplicate records, or invalid values.

Before giving the dataset to the Machine Learning algorithm, we must clean it.

Typical preprocessing tasks include:

- Loading the CSV file.
- Checking the number of records.
- Removing duplicate rows.
- Handling missing values.
- Validating numeric columns.
- Selecting only useful columns.
- Separating input features and target values.

Data preprocessing is one of the most important steps because a Machine Learning model is only as good as the quality of the data used to train it.

6. Feature Selection and Target Variable

The Machine Learning model learns using **input features** and predicts a **target variable**.

For our project:

Input Features (X):

- Attendance
- Study Hours
- Previous Marks
- Assignment Marks
- Internal Marks

These variables influence student performance.

Target Variable (Y):

- Final Marks

The model studies the relationship between these input features and the final examination marks.

Choosing the correct features is extremely important because irrelevant features reduce prediction accuracy.

7. Train-Test Split

After preparing the dataset, we divide it into two parts:

Training Dataset (80%)

The model learns from this data.

Testing Dataset (20%)

This data is never shown during training. It is used to evaluate whether the model can predict marks for new students.

If we test the model using the same data used for training, the evaluation will not be reliable. Therefore, every Machine Learning project separates training and testing datasets.

8. Building the Linear Regression Model

Linear Regression is one of the simplest supervised Machine Learning algorithms.

It identifies the mathematical relationship between the input features and the target variable.

During training, the algorithm analyzes historical student records and calculates coefficients that best fit the data.

Once training is complete, the model can estimate the final marks of students it has never seen before.

The trained model is stored in memory and is ready for evaluation.

9. Model Evaluation

Training a model is not enough. We must measure how accurately it predicts.

We compare the predicted marks with the actual marks in the testing dataset.

Common evaluation metrics include:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

A good model produces a low prediction error and a high R^2 score.

Evaluation helps us determine whether the model is reliable enough for practical use.

10. Saving the Model

Training a Machine Learning model every time the application starts is inefficient.

Instead, once the model has been trained successfully, we save it to disk using **Joblib**.

This creates a .pkl file that contains the trained model.

Whenever the application starts, it loads this saved model instead of training again.

This approach reduces application startup time and follows industry best practices.

11. Making Predictions

Prediction is the final purpose of the Machine Learning model.

When a user enters:

- Attendance
- Study Hours
- Previous Marks
- Assignment Marks
- Internal Marks

the application loads the saved model, processes the input values, and calculates the predicted final examination marks.

The prediction is displayed instantly to the user.

12. Building the Streamlit Application

Machine Learning models usually run in the background, but users need a simple interface to interact with them.

We will develop a Streamlit web application where users can enter student information through input fields.

When the **Predict** button is clicked, Streamlit sends the values to the prediction function, which loads the saved model, predicts the marks, and displays the result.

This converts our Machine Learning model into a user-friendly web application.

13. Testing the Application

Before deploying the project, we verify that it works correctly.

Testing includes:

- Checking valid inputs.
- Checking invalid inputs.
- Verifying prediction accuracy.
- Confirming the model loads successfully.
- Ensuring no runtime errors occur.

Testing ensures the application is stable and reliable.

14. Project Deployment

The final stage is deployment.

Deployment makes the application available for teachers and students to use through a web browser.

The project includes:

- Source Code
- Dataset
- Trained Model
- Streamlit Application
- Requirements File
- Documentation